

Gestiona Gateway API Documentation

Versão 1.6.1

Index

Models

- UploadDocumentRequest
- CreateProcessRequest
- RelatedProcessesRequest
- GatewayResponse
- UploadDocumentResult
- UploadDocumentError
- ThirdResult
- ThirdError
- ActivityItem
- ProcedureItem
- ActivityError
- ProcessResult
- ProcessError
- ProcessThirdsResult
- ProcessThirdsError
- ProcessDocumentItem
- ProcessDocumentsError
- ProcessAssigneeUserRequest
- ProcessAssigneeUserResult
- ProcessAssigneeGroupResult
- QueueConnectorMessageResult
- SendQueueConnectorResponseRequest
- QueueConnectorResponseResult
- QueueError
- Download Success Output

Shared

- Request headers

Endpoints

- 1. POST /processes
- 2. POST /processes/related
- 3. DELETE /processes/{process_id}/related/{related_process_id}
- 4. GET /processes?process_number=<numero>
- 5. POST /processes/documents?process_number=<numero>
- 6. POST /processes/{process_id}/documents
- 7. POST /processes/documents/{folder_id}?process_number=<numero>
- 8. POST /processes/{process_id}/documents/{folder_id}
- 9. GET /processes/thirds?process_number=<numero>
- 10. GET /processes/{process_id}/thirds
- 11. GET /processes/{process_id}/documents

- 12. GET /processes/{process_id}/documents/{document_id}
- 13. GET /processes/assignees/users
- 14. GET /processes/assignees/groups
- 15. GET /activities
- 16. GET /activities/{activity_id}/procedures
- 17. GET /documents/{document_id}
- 18. GET /thirds?nif=<nif>
- 19. GET /thirds/{third_id}
- 20. GET /queues/connectors/{connector_name}
- 21. GET /queues/connectors/{connector_name}/{message_id}
- 22. POST /queues/connectors/{connector_name}/{message_id}

Models

UploadDocumentRequest

Used as the request body for both upload endpoints.

```
{
  "operationId": "string | null",
  "id": "string | null",
  "name": "string | null",
  "fileName": "string | null",
  "documentSourceType": "string | null",
  "url": "string | null",
  "content": "string | null"
}
```

Field notes

- `documentSourceType` Expected values in the current implementation are `DIGITAL`, `EXTERNAL_URL`, and `FOLDER`.
- `fileName` Used for `DIGITAL` uploads when the file is read from local storage.
- `content` Base64-encoded file content for `DIGITAL` uploads.
- `url` External URL used when `documentSourceType` is `EXTERNAL_URL`.

CreateProcessRequest

Used as the request body for POST `/processes`.

```
{
  "activityId": "string",
  "procedureId": "string",
  "userId": "string",
  "groupId": "string",
  "freeSubject": "string"
}
```

Field notes

- `activityId` Sent upstream as the Gestiona catalog procedure id in `catalog-2015/procedures/{activityId}`.
- `procedureId` Sent upstream as the Gestiona external procedure id in `external-procedures/{procedureId}`.
- `userId` Used to build the file-opening initial assignation link: `{GestionaApiBaseUrl}/users/{userId}`.
- `groupId` Used to build the file-opening management unit group link: `{GestionaApiBaseUrl}/groups/{groupId}`.
- `freeSubject` Sent upstream as `free_title`.

RelatedProcessesRequest

Used as the request body for POST `/processes/related`.

```
{
  "id1": "string",
  "id2": "string"
}
```

Field notes

- `id1` The Gestiona file id that will receive the related-file link.
- `id2` The Gestiona file id to link as a related file.

GatewayResponse

Used as the response envelope for gateway success and error responses.

```
{
  "operationId": "string | null",
  "success": true,
  "result": {}
}
```

UploadDocumentResult

Used inside `GatewayResponse.result` on upload success.

```
{
  "id": "string",
  "processId": "string",
  "creation_date": "string",
  "modification_date": "string"
}
```

Field notes

- `id` The API returns the created Gestiona entity id. If the upstream create response does not include `id`, the service resolves it from the last path segment of the `self` link in the upstream `Links` collection.

UploadDocumentError

Used inside `GatewayResponse.result` on upload errors and download errors.

```
{
  "code": 400,
  "name": "Bad Request",
  "kind": "Validation",
  "message": "string"
}
```

Possible kind values for upload

- Configuration
- Validation
- NotFound
- Upstream

Possible kind values for download

- Configuration
- Validation
- NotFound
- Upstream

ThirdResult

Used inside `GatewayResponse.result` on third lookup success.

```
{
  "full_name": "string | null",
  "first_name": "string | null",
  "second_surname": "string | null",
  "nif_country": "string | null",
  "id": "string | null",
  "nif": "string | null",
  "type": "string | null",
  "email": "string | null",
  "mobile": "string | null",
  "nif_type": "string | null",
  "address": "string | null",
  "number": "string | null",
  "zip_code": "string | null",
  "province": "string | null",
  "country": "string | null",
  "type_of_road": "string | null",
  "zone": "string | null",
  "parish_code": "string | null"
}
```

Field notes

- Address fields are obtained from Gestiona GET `/thirds/{third_id}/default-address` after the base third is retrieved.

ThirdError

Used inside `GatewayResponse.result` on third lookup errors.

```
{
  "code": 400,
  "name": "Bad Request",
  "kind": "Validation",
  "message": "string"
}
```

Possible kind values for third lookup

- Configuration
- Validation
- NotFound
- Upstream

ActivityItem

Used for each item in `GatewayResponse.result` returned by GET `/activities`.

```
{
  "id": "string | null",
  "name": "string | null"
}
```

ProcedureItem

Used for each item in `GatewayResponse.result` returned by GET `/activities/{activity_id}/procedures`.

```
{
  "id": "string | null",
  "name": "string | null",
  "activityId": "string | null"
}
```

Field notes

- name Mapped from the upstream external procedure title field.
- activityId Copied from the activity_id route parameter.

ActivityError

Used inside `GatewayResponse.result` on activity and procedure lookup errors.

```
{
  "code": 400,
  "name": "Bad Request",
  "kind": "Upstream",
  "message": "string"
}
```

Possible kind values

- Configuration
- Upstream

ProcessResult

Used inside `GatewayResponse.result` on process lookup success.

```
{
  "id": "string",
  "processNumber": "string"
}
```

Field notes

- `id` The resolved Gestiona file id.
- `processNumber` The process number used to resolve the file id.

ProcessError

Used inside `GatewayResponse.result` on process lookup errors.

```
{
  "code": 400,
  "name": "Bad Request",
  "kind": "Validation",
  "message": "string"
}
```

Possible kind values for process lookup

- Configuration
- Validation
- NotFound
- Upstream

ProcessThirdsResult

Used inside `GatewayResponse.result` on process thirds lookup success.

```
{
  "processId": "string",
  "thirds": "third-id-1;third-id-2"
}
```

Field notes

- `thirds` Semicolon-separated third ids extracted from each upstream `rel: third` link.

ProcessThirdsError

Used inside `GatewayResponse.result` on process thirds lookup errors.

```
{
  "code": 400,
  "name": "Bad Request",
  "kind": "Validation",
  "message": "string"
}
```

Possible kind values for process thirds lookup

- Configuration
- Validation
- NotFound
- Upstream

ProcessDocumentItem

Used for each item in `GatewayResponse.result` returned by the process document-listing endpoints.

```
{
  "type": "DOC",
  "name": "POC_SIGMA_Gestiona",
  "id": "f5e0364b-9951-449d-8d38-34a6cbfec4d3"
}
```

Field notes

- type The upstream item type, such as DOC or FOLDER.
- name The upstream rel value.
- id The last path segment of the upstream href.

ProcessDocumentsError

Used inside `GatewayResponse.result` when a process document-listing request fails.

```
{
  "code": 400,
  "name": "Bad Request",
  "kind": "Validation",
  "message": "string"
}
```

Possible kind values

- Configuration
- Validation
- NotFound
- Upstream

ProcessAssigneeUserRequest

Used as the JSON request body for GET `/processes/assignees/users`.

```
{
  "username": "string"
}
```

ProcessAssigneeUserResult

Used inside `GatewayResponse.result` on assignee user lookup success.

```
{
  "id": "string | null",
  "username": "string | null",
  "name": "string | null"
}
```

ProcessAssigneeGroupResult

Used for each item in `GatewayResponse.result` returned by GET `/processes/assignees/groups`.

```
{
  "id": "string | null",
  "name": "string | null"
}
```

QueueConnectorMessageResult

Used inside `GatewayResponse.result` on queue connector message success. For GET `/queues/connectors/{connector_name}`, result is an array of this model.

```
{
  "message_id": "string | null",
  "date_signed": "string | null"
}
```

Field notes

- `message_id` Mapped from the upstream queue message payload.target field.
- `date_signed` Mapped from the upstream queue message entry field after formatting with `DateTimeHelpers.FormatUnixTimestamp`.

SendQueueConnectorResponseRequest

Used as the JSON request body for POST `/queues/connectors/{connector_name}/{message_id}`.

```
{
  "result_success": "string",
  "message": "string | null"
}
```

Field notes

- `result_success` Required. Sent upstream as `result_success`.
- `message` Optional. When null or not present, it is not sent in the upstream JSON payload.

QueueConnectorResponseResult

Used inside `GatewayResponse.result` when a connector response is sent successfully.

```
{
  "connector_name": "string",
  "message_id": "string"
}
```

QueueError

Used inside `GatewayResponse.result` on queue connector errors.

```
{
  "code": 400,
  "name": "Bad Request",
  "kind": "Validation",
  "message": "string"
}
```

Possible kind values for queue connector message

- Configuration
- Validation
- NotFound
- NoActiveSubscription
- Upstream

Download Success Output

The download endpoint does not return JSON on success. It returns the raw document bytes in the response body.

Relevant response characteristics

- Body: binary file content
- Content-Type: document MIME type returned by Gestiona, or application/octet-stream as fallback
- Content-Disposition: attachment, with automatic download filename

The underlying DLL model used by the service layer is

```
{
  "documentId": "string",
  "fileName": "string | null",
  "contentType": "string | null",
  "storageSize": 0,
  "storageExtension": "string | null",
  "storageMimeType": "string | null",
  "storageMd5": "string | null",
  "storageSha1": "string | null",
  "storageSha512": "string | null",
  "content": "byte[]"
}
```

Request headers

All endpoints can optionally receive:

- `X-User -Access-Token` When present and not blank, the gateway uses this value as the upstream `Gestiona X-Gestiona-Access-Token`.

If `X-User -Access-Token` is absent or blank, the gateway uses the configured token from `Gestiona:AccessToken`.

The queue connector endpoints are exceptions: they always use the configured token from `Gestiona:AccessToken` and ignore the request header token.

Endpoints

1. POST /processes

Creates a new Gestiona process by creating the upstream file from the catalog/external procedure pair and then opening that file through the file-open link returned by Gestiona.

Query parameters

- `operationId` optional

Request body model

- `CreateProcessRequest`

Request body example

```
{
  "activityId": "82722c9b-cecc-4299-8a7b-ce5abeb8170b",
  "procedureId": "external-procedure-id",
  "userId": "3f18aa72-4091-4a50-8210-6c05ca234647",
  "groupId": "2bb7ddb2-a870-470d-ae87-3d4b2d3dd4af",
  "freeSubject": "Mais um processo criado a partir da API"
}
```

Upstream calls

1. POST /catalog-2015/procedures/{activityId}/external-procedures/{procedureId}/create-file
2. GET /files/{process_id}/selectable-titles
 - `process_id` is extracted from the file-open link returned by create-file.
 - If `selectable_titles` contains at least one non-blank value, the first value is used as `selectable_title` in the file-open request.
3. POST {file-open href returned by create-file}
 - Content-Type: application/vnd.gestiona.file-opening+json; version=1
 - `entry_date` is copied from the create-file response.
 - `free_title` is copied from `freeSubject`.
 - `selectable_title` is included only when a selectable title was returned by GET /files/{process_id}/selectable-titles.
 - `initial_assignment[0].href` is {GestionaApiBaseUrl}/users/{userId}.
 - `links[0].href` is {GestionaApiBaseUrl}/groups/{groupId} with rel equal to management-unit-group.

Success response

- HTTP 200 OK
- Body model: `GatewayResponse`
- result shape: `ProcessResult`

Success example

```
{
  "operationId": "op-01",
  "success": true,
  "result": {
    "Id": "a7a43429-a82c-4245-9f50-f1e853905a99",
    "processNumber": "16/2026"
  }
}
```

Error response

- HTTP 400, 500, or propagated upstream status code
- Body model: `GatewayResponse`

- `result` shape: `ProcessError`

Notes

- The first upstream response must include `entry_date` and a link with `rel` equal to `file-open`.
- The `file-open` link must include the created Gestion process id so the gateway can call `GET /files/{process_id}/selectable-titles`.
- The `selectable-titles` response shape is `{"required": true, "selectable_titles": ["Teste 1", "Teste 2"]}`.
- If `selectable_titles` is null, empty, or contains only blank values, `selectable_title` is omitted from the `file-open` request.
- The `file-open` upstream response must include both `id` and `code`.
- `ProcessResult.Id` is mapped from the `file-open` upstream response `id`.
- `ProcessResult.processNumber` is mapped from the `file-open` upstream response `code`.

2. POST /processes/related

Creates a related-file link between two Gestion process files.

Route parameters

- none

Query parameters

- `operationId` optional

Request body model

- `RelatedProcessesRequest`

Request body example

```
{
  "id1": "30bcb012-47e2-4e7e-92e0-a0f7278b52b8",
  "id2": "a7a43429-a82c-4245-9f50-f1e853905a99"
}
```

Upstream calls

1. `POST /files/{id1}/related-files`
 - `Content-Type: application/vnd.gestiona.links+json`
 - Sends a `links` array with one item where `rel` is `related-files`.
 - The link `href` is `{GestionApiBaseUrl}/files/{id2}`.

Success response

- HTTP 200 OK
- Body model: `GatewayResponse`
- `result` shape: `RelatedProcessesRequest`

Success example

```
{
  "operationId": "op-01",
  "success": true,
  "result": {
    "id1": "30bcb012-47e2-4e7e-92e0-a0f7278b52b8",
    "id2": "a7a43429-a82c-4245-9f50-f1e853905a99"
  }
}
```

Error response

- HTTP 400, 404, 500, or propagated upstream status code
- Body model: GatewayResponse
- result shape: ProcessError

Notes

- If the request body is missing, the endpoint returns HTTP 400.
- If id1 or id2 is empty or whitespace, the endpoint returns HTTP 400.
- If Postman sends an unresolved variable such as {{id1}} or {{id2}}, the endpoint returns HTTP 400.

3. DELETE /processes/{process_id}/related/{related_process_id}

Deletes a related-file link between two Gestion process files.

Route parameters

- process_id required
- related_process_id required

Query parameters

- operationId optional

Request body model

- none

Upstream calls

1. DELETE /files/{process_id}/related-files/{related_process_id}

Success response

- HTTP 200 OK
- Body model: GatewayResponse
- result shape: empty object

Success example

```
{
  "operationId": "op-01",
  "success": true,
  "result": {}
}
```

Error response

- HTTP 400, 404, 500, or propagated upstream status code
- Body model: GatewayResponse
- result shape: ProcessError

Notes

- If process_id or related_process_id is empty or whitespace, the endpoint returns HTTP 400.
- If Postman sends an unresolved variable such as {{process_id}} or {{related_process_id}}, the endpoint returns HTTP 400.

4. GET /processes?process_number=<numero>

Resolves the Gestion file id associated with process_number.

Route parameters

- none

Query parameters

- process_number required
- operationId optional

Request body model

- none

Upstream calls

1. GET /files with filter exact_code = process_number

Success response

- HTTP 200 OK
- Body model: GatewayResponse
- result shape: ProcessResult

Success example

```
{
  "operationId": "op-01",
  "success": true,
  "result": {
    "Id": "30bcb012-47e2-4e7e-92e0-a0f7278b52b8",
    "processNumber": "PROC-2026-001"
  }
}
```

Error response

- HTTP 400, 404, 500, or propagated upstream status code
- Body model: GatewayResponse
- result shape: ProcessError

Notes

- Id is the resolved Gestiona file id, not the original process_number
- If no Gestiona file is found for process_number, the endpoint returns HTTP 404
- If process_number is empty or whitespace, the endpoint returns HTTP 400
- If Postman sends an unresolved variable such as {{process_number}}, the endpoint returns HTTP 400

5. POST /processes/documents?process_number=<numero>

Creates a document by resolving the target Gestiona process_id (file id in gestiona) from the query parameter process_number.

Query parameters

- process_number required

Request body model

- UploadDocumentRequest

Request body examples

DIGITAL

```
{
  "operationId": "op-123",
  "name": "Contrato",
  "fileName": "contrato.pdf",
  "documentSourceType": "DIGITAL",
  "content": "JVBERi0xLjQKJ..."
}
```

FOLDER

```
{
  "operationId": "op-123",
  "name": "Expediente 2026",
  "documentSourceType": "FOLDER"
}
```

EXTERNAL_URL

```
{
  "operationId": "op-123",
  "name": "Referencia externa",
  "documentSourceType": "EXTERNAL_URL",
  "url": "https://example.com/documento/123"
}
```

Upstream calls

1. GET /files with filter exact_code = process_number
2. For DIGITAL requests:
 - POST /uploads
 - PUT {upload_location}
 - POST /files/{resolved_process_id}/documents-and-folders
3. For FOLDER requests:
 - POST /files/{resolved_process_id}/documents-and-folders
4. For EXTERNAL_URL requests:
 - POST /files/{resolved_process_id}/documents-and-folders

Success response

- HTTP 200 OK
- Body model: GatewayResponse
- result shape: UploadDocumentResult

Success example

```
{
  "operationId": "op-123",
  "success": true,
  "result": {
    "id": "document-id",
    "processId": "file-id",
    "creation_date": "2026-05-08 10:00:00",
    "modification_date": "2026-05-08 10:00:00"
  }
}
```

Error response

- HTTP 400, 404, 500, or propagated upstream status code
- Body model: GatewayResponse
- result shape: UploadDocumentError

Error example

```
{
  "operationId": "op-123",
  "success": false,
  "result": {
    "code": 400,
    "name": "Bad Request",
    "kind": "Validation",
    "message": "process_number query parameter is required."
  }
}
```

6. POST /processes/{process_id}/documents

Creates a document directly in the Gestion file identified by `process_id`.

Route parameters

- `process_id` required

Request body model

- UploadDocumentRequest

Request body examples

DIGITAL

```
{
  "operationId": "op-123",
  "name": "Contrato",
  "fileName": "contrato.pdf",
  "documentSourceType": "DIGITAL",
  "content": "JVBERi0xLjQKJ..."
}
```

FOLDER

```
{
  "operationId": "op-123",
  "name": "Expediente 2026",
  "documentSourceType": "FOLDER"
}
```

EXTERNAL_URL

```
{
  "operationId": "op-123",
  "name": "Referencia externa",
  "documentSourceType": "EXTERNAL_URL",
  "url": "https://example.com/documento/123"
}
```

Upstream calls

1. For DIGITAL requests:

- POST /uploads
- PUT {upload_location}
- POST /files/{process_id}/documents-and-folders

2. For FOLDER requests:

- POST /files/{process_id}/documents-and-folders

3. For EXTERNAL_URL requests:

- POST /files/{process_id}/documents-and-folders

Success response

- HTTP 200 OK
- Body model: GatewayResponse

- result shape: UploadDocumentResult

Error response

- HTTP 400, 404, 500, or propagated upstream status code
- Body model: GatewayResponse
- result shape: UploadDocumentError

Notes

- For DIGITAL uploads, either fileName or content must be provided.
- If both fileName and content are provided, the current implementation uses content.
- On successful create operations, result.id may come either from the upstream id field or, when that field is missing, from the last segment of the upstream self link.

7. POST /processes/documents/{folder_id}?process_number=<numero>

Creates a document inside the Gestion folder identified by folder_id, after resolving the target Gestion file from the query parameter process_number.

Route parameters

- folder_id required

Query parameters

- process_number required

Request body model

- UploadDocumentRequest

Request body examples

DIGITAL

```
{
  "operationId": "op-123",
  "name": "Contrato",
  "fileName": "contrato.pdf",
  "documentSourceType": "DIGITAL",
  "content": "JVBERi0xLjQKJ..."
}
```

FOLDER

```
{
  "operationId": "op-123",
  "name": "Expediente 2026",
  "documentSourceType": "FOLDER"
}
```

EXTERNAL_URL

```
{
  "operationId": "op-123",
  "name": "Referencia externa",
  "documentSourceType": "EXTERNAL_URL",
  "url": "https://example.com/documento/123"
}
```

Upstream calls

1. GET /files with filter exact_code = process_number
2. For DIGITAL requests:
 - POST /uploads
 - PUT {upload_location}

- `POST /files/{resolved_process_id}/documents-and-folders/{folder_id}`
3. For FOLDER requests:
- `POST /files/{resolved_process_id}/documents-and-folders/{folder_id}`
4. For EXTERNAL_URL requests:
- `POST /files/{resolved_process_id}/documents-and-folders/{folder_id}`

Success response

- HTTP 200 OK
- Body model: `GatewayResponse`
- result shape: `UploadDocumentResult`

Error response

- HTTP 400, 404, 500, or propagated upstream status code
- Body model: `GatewayResponse`
- result shape: `UploadDocumentError`

Notes

- This route uses the same document creation flows as the process-number route, but targets the upstream Gestiona endpoint with the folder id in the last path segment.
- For DIGITAL uploads, either `fileName` or `content` must be provided.
- If both `fileName` and `content` are provided, the current implementation uses `content`.
- On successful create operations, `result.id` may come either from the upstream `id` field or, when that field is missing, from the last segment of the upstream `self` link.

8. `POST /processes/{process_id}/documents/{folder_id}`

Creates a document directly inside the Gestiona folder identified by `folder_id`, under the file identified by `process_id`.

Route parameters

- `process_id` required
- `folder_id` required

Request body model

- `UploadDocumentRequest`

Request body examples

DIGITAL

```
{
  "operationId": "op-123",
  "name": "Contrato",
  "fileName": "contrato.pdf",
  "documentSourceType": "DIGITAL",
  "content": "JVBERi0xLjQKJ..."
}
```

FOLDER

```
{
  "operationId": "op-123",
  "name": "Expediente 2026",
  "documentSourceType": "FOLDER"
}
```

EXTERNAL_URL

```
{
  "operationId": "op-123",
  "name": "Referencia externa",
  "documentSourceType": "EXTERNAL_URL",
  "url": "https://example.com/documento/123"
}
```

Upstream calls

1. For DIGITAL requests:
 - POST /uploads
 - PUT {upload_location}
 - POST /files/{process_id}/documents-and-folders/{folder_id}
2. For FOLDER requests:
 - POST /files/{process_id}/documents-and-folders/{folder_id}
3. For EXTERNAL_URL requests:
 - POST /files/{process_id}/documents-and-folders/{folder_id}

Success response

- HTTP 200 OK
- Body model: GatewayResponse
- result shape: UploadDocumentResult

Error response

- HTTP 400, 404, 500, or propagated upstream status code
- Body model: GatewayResponse
- result shape: UploadDocumentError

Notes

- This route uses the same document creation flows as the file-level route, but targets the upstream Gestion endpoint with the folder id in the last path segment.
- For DIGITAL uploads, either fileName or content must be provided.
- If both fileName and content are provided, the current implementation uses content.
- On successful create operations, result.id may come either from the upstream id field or, when that field is missing, from the last segment of the upstream self link.

9. GET /processes/thirds?process_number=<numero>

Gets the third identifiers associated with a Gestion process file resolved from process_number.

Route parameters

- none

Query parameters

- process_number required
- operationId optional

Request body model

- none

Upstream calls

1. GET /files with filter exact_code = process_number
2. GET /files/{resolved_process_id}/thirdparties

Success response

- HTTP 200 OK
- Body model: GatewayResponse
- result shape: ProcessThirdsResult

Success example

```
{
  "operationId": "op-01",
  "success": true,
  "result": {
    "processId": "30bcb012-47e2-4e7e-92e0-a0f7278b52b8",
    "thirds": "3aeff9c7-a865-4f1a-9cd6-47993b423873;4b18954c-b66c-4e55-af6d-acf6a2c7aaa3;ece5762f-ae00-4da4-a869-a"
  }
}
```

Error response

- HTTP 400, 404, 500, or propagated upstream status code
- Body model: GatewayResponse
- result shape: ProcessThirdsError

Notes

- The returned processId is the resolved Gestiona file id, not the original process_number
- If no Gestiona file is found for process_number, the endpoint returns HTTP 404
- If process_number is empty or whitespace, the endpoint returns HTTP 400
- If Postman sends an unresolved variable such as {{process_number}}, the endpoint returns HTTP 400

10. GET /processes/{process_id}/thirds

Gets the third identifiers associated with a Gestiona process file.

Route parameters

- process_id required

Query parameters

- operationId optional

Request body model

- none

Upstream calls

1. GET /files/{process_id}/thirdparties

Success response

- HTTP 200 OK
- Body model: GatewayResponse
- result shape: ProcessThirdsResult

Success example

```
{
  "operationId": "op-01",
  "success": true,
  "result": {
    "processId": "30bcb012-47e2-4e7e-92e0-a0f7278b52b8",
    "thirds": "3aeff9c7-a865-4f1a-9cd6-47993b423873;4b18954c-b66c-4e55-af6d-acf6a2c7aaa3;ece5762f-ae00-4da4-a869-a"
  }
}
```

Error response

- HTTP 400, 404, 500, or propagated upstream status code
- Body model: GatewayResponse
- result shape: ProcessThirdsError

Notes

- The service reads the upstream content array and, for each item, finds the link where rel is third
- The third id is extracted from the last segment of that link's href
- The returned thirds field joins all extracted third ids with semicolons
- If process_id is empty or whitespace, the endpoint returns HTTP 400
- If Postman sends an unresolved variable such as {{process_id}}, the endpoint returns HTTP 400

11. GET /processes/{process_id}/documents

Gets the documents and folders at the root of a Gestion process file.

Route parameters

- process_id required

Query parameters

- operationId optional

Request body model

- none

Upstream calls

1. GET /files/{process_id}/documents-and-folders

Success response

- HTTP 200 OK
- Body model: GatewayResponse
- result shape: array of ProcessDocumentItem

Success example

```
{
  "operationId": "op-01",
  "success": true,
  "result": [
    {
      "type": "DOC",
      "name": "POC_SIGMA_Gestion",
      "id": "f5e0364b-9951-449d-8d38-34a6cbfec4d3"
    },
    {
      "type": "FOLDER",
      "name": "xxxx",
      "id": "12fb9b74-2111-417f-ae4f-c2b7fe2976f7"
    }
  ]
}
```

Error response

- HTTP 400, 404, 500, or propagated upstream status code
- Body model: GatewayResponse
- result shape: ProcessDocumentsError

Notes

- Both DOC and FOLDER entries from the upstream content array are returned.
- Each upstream rel is mapped to name.
- Each id is extracted from the last path segment of the upstream href.
- If process_id is empty or whitespace, the endpoint returns HTTP 400.
- If Postman sends an unresolved variable such as {{process_id}}, the endpoint returns HTTP 400.

12. GET /processes/{process_id}/documents/{document_id}

Gets the documents and folders contained inside the specified Gestion document or folder.

Route parameters

- process_id required
- document_id required

Query parameters

- operationId optional

Request body model

- none

Upstream calls

1. GET /files/{process_id}/documents-and-folders/{document_id}

Success response

- HTTP 200 OK
- Body model: GatewayResponse
- result shape: array of ProcessDocumentItem

Success example

```
{
  "operationId": "op-01",
  "success": true,
  "result": [
    {
      "type": "DOC",
      "name": "Nested document",
      "id": "347d4226-093f-412d-8376-e36d36374d13"
    }
  ]
}
```

Error response

- HTTP 400, 404, 500, or propagated upstream status code
- Body model: GatewayResponse
- result shape: ProcessDocumentsError

Notes

- The response mapping is identical to GET /processes/{process_id}/documents.
- Both DOC and FOLDER entries from the upstream content array are returned.
- If either route parameter is empty or whitespace, the endpoint returns HTTP 400.
- If Postman sends an unresolved {{process_id}} or {{document_id}} variable, the endpoint returns HTTP 400.

13. GET /processes/assignees/users

Gets the first Gestiona assignee user matching the provided username.

Route parameters

- none

Query parameters

- `operationId` optional
- `username` optional. When present and not blank, this value is used instead of the JSON request body.

Request body model

- `ProcessAssigneeUserRequest`, required only when username is not provided as a query parameter.

Request body example

```
{
  "username": "081847637"
}
```

Upstream calls

1. GET /files/assignees/users

- Request body: `{"username": "081847637"}`
- Content-Type: `application/vnd.gestiona.filter.assignees+json`

Success response

- HTTP 200 OK
- Body model: `GatewayResponse`
- result shape: `ProcessAssigneeUserResult`

Success example

```
{
  "operationId": "op-01",
  "success": true,
  "result": {
    "id": "8be7a78b-787a-4061-a11c-1bfcd2d627a",
    "username": "081847637",
    "name": "Luis Silva"
  }
}
```

Error response

- HTTP 400, 404, 415, 500, 502, or propagated upstream status code
- Body model: `GatewayResponse`
- result shape: `ProcessError`

Notes

- When `username` is provided in the query string, no JSON request body is required.
- When `username` is not provided in the query string, the endpoint requires Content-Type: `application/json`.
- When using the JSON-body path, the request body must be valid JSON.
- The service returns the first item from the upstream content array.
- If no assignee user is found, the endpoint returns HTTP 404.

14. GET /processes/assignees/groups

Gets the Gestiona assignee groups available for process assignment.

Route parameters

- none

Query parameters

- `operationId` optional

Request body model

- none

Upstream calls

1. GET `/files/assignees/groups`

Success response

- HTTP 200 OK
- Body model: `GatewayResponse`
- result shape: array of `ProcessAssigneeGroupResult`

Success example

```
{
  "operationId": "op-01",
  "success": true,
  "result": [
    {
      "id": "43f83662-bb73-4c98-915a-de90219036f6",
      "name": "100. Exemplo"
    }
  ]
}
```

Error response

- HTTP 404, 500, 502, or propagated upstream status code
- Body model: `GatewayResponse`
- result shape: `ProcessError`

Notes

- The service returns the upstream content array.
- Only `id` and `name` are exposed in each result item.
- If the upstream response contains no content, the endpoint returns an empty array.

15. GET `/activities`

Gets the activities available in the Gestiona catalog.

Route parameters

- none

Query parameters

- `operationId` optional

Request body model

- none

Upstream calls

1. GET `/catalog-2015/procedures`

Success response

- HTTP 200 OK
- Body model: GatewayResponse
- result shape: array of ActivityItem

Success example

```
{
  "operationId": "op-01",
  "success": true,
  "result": [
    {
      "id": "82722c9b-cecc-4299-8a7b-ce5abeb8170b",
      "name": "Atividade exemplo"
    }
  ]
}
```

Error response

- HTTP 500, 502, or propagated upstream status code
- Body model: GatewayResponse
- result shape: ActivityError

Notes

- The service returns the upstream content array.
- If the upstream response contains no content, the endpoint returns an empty array.

16. GET /activities/{activity_id}/procedures

Gets the external procedures available for a Gestiona activity.

Route parameters

- activity_id required

Query parameters

- operationId optional

Request body model

- none

Upstream calls

1. GET /catalog-2015/procedures/{activity_id}/external-procedures

Success response

- HTTP 200 OK
- Body model: GatewayResponse
- result shape: array of ProcedureItem

Success example

```
{
  "operationId": "op-01",
  "success": true,
  "result": [
    {
      "id": "external-procedure-id",
      "name": "Procedimento exemplo",
      "activityId": "82722c9b-cecc-4299-8a7b-ce5abeb8170b"
    }
  ]
}
```

Error response

- HTTP 500, 502, or propagated upstream status code
- Body model: GatewayResponse
- result shape: ActivityError

Notes

- The gateway maps each upstream external procedure title to response field name.
- If the upstream response contains no content, the endpoint returns an empty array.

17. GET /documents/{document_id}

Downloads a document from Gestiona. This is an absolute route and is not prefixed by /processes.

Route parameters

- document_id required

Query parameters

- operationId optional

Request body model

- none

Upstream calls

1. GET /content/small/documentinstances/{document_id}

Success response

- HTTP 200 OK
- Body: raw binary document content
- Headers:
 - Content-Type: document MIME type returned by Gestiona, or application/octet-stream as fallback
 - Content-Disposition: attachment; filename=...
 - X-Operation-Id: present when operationId is provided in the request
 - X-Storage-Extension: present when the upstream document metadata includes a storage extension

Download filename resolution

The controller chooses the download filename in this order:

1. document.fileName
2. document.documentId + "." + document.storageExtension
3. document.documentId

Validation behavior

- If document_id is empty or whitespace, the endpoint returns HTTP 400

- If Postman sends an unresolved variable such as `{{document_id}}`, the endpoint returns HTTP 400

Error response

- HTTP 400, 404, 500, or propagated upstream status code
- Body model: GatewayResponse
- result shape: UploadDocumentError

Error example

```
{
  "operationId": "op-123",
  "success": false,
  "result": {
    "code": 404,
    "name": "Not Found",
    "kind": "NotFound",
    "message": "Failed to download document from Gestiona: 12345."
  }
}
```

Notes

- Successful downloads do not return JSON
- Download errors reuse the same GatewayResponse envelope as upload errors
- When operationId is provided in the download request, it is echoed back:
 - in the error JSON body on failure
 - in the X-Operation-Id response header on success
- When the upstream response includes document storage extension metadata, it is exposed in the X-Storage-Extension response header

18. GET /thirds?nif=<nif>

Gets a third from Gestiona by resolving the third id from a NIF, then enriches it with the default address.

Route parameters

- none

Query parameters

- nif required
- operationId optional

Request body model

- none

Upstream calls

1. GET /thirds with body `{"nif":"nif"}` and Content-Type: application/vnd.gestiona.filter.thirds+json
2. GET /thirds/{resolved_third_id}
3. GET /thirds/{resolved_third_id}/default-address

Success response

- HTTP 200 OK
- Body model: GatewayResponse
- result shape: ThirdResult

Success example

```
{
  "operationId": "op-01",
  "success": true,
  "result": {
    "full_name": "Luis Silva Fernandes",
    "first_name": "Luis",
    "second_surname": "Fernandes",
    "nif_country": "ESP",
    "id": "4b18954c-b66c-4e55-af6d-acf6a2c7aaa3",
    "nif": "196510880",
    "type": "PHISIC",
    "email": "luis.mcf.silva@gmail.com",
    "mobile": "913347827",
    "nif_type": "OWN",
    "address": "Rua das Cancelas",
    "number": "184",
    "zip_code": "4440368",
    "province": "PORTO",
    "country": "Portugal",
    "type_of_road": "CL",
    "zone": "string | null",
    "parish_code": "string | null"
  }
}
```

Error response

- HTTP 400, 404, 500, or propagated upstream status code
- Body model: GatewayResponse
- result shape: ThirdError

Notes

- The upstream NIF filter response must contain exactly one item in content
- The third id is extracted from the id field of that single item
- If nif is empty or whitespace, the endpoint returns HTTP 400
- If Postman sends an unresolved variable such as {{nif}}, the endpoint returns HTTP 400

19. GET /thirds/{third_id}

Gets a third from Gestiona and enriches it with the default address.

Route parameters

- third_id required

Query parameters

- operationId optional

Request body model

- none

Upstream calls

1. GET /thirds/{third_id}
2. GET /thirds/{third_id}/default-address

Success response

- HTTP 200 OK
- Body model: GatewayResponse
- result shape: ThirdResult

Success example

```
{
  "operationId": "op-01",
  "success": true,
  "result": {
    "full_name": "Leonor Ranito Silva",
    "first_name": "Leonor",
    "second_surname": "Silva",
    "nif_country": "PT",
    "id": "3aeff9c7-a865-4f1a-9cd6-47993b423873",
    "nif": "211211211",
    "type": "PHISIC",
    "email": "leonor.silva@gmail.com",
    "mobile": "913344671",
    "nif_type": "OWN",
    "address": "Rua das Cancelas",
    "number": "184",
    "zip_code": "4440368",
    "province": "PORTO",
    "country": "Portugal",
    "type_of_road": "CL",
    "zone": "string | null",
    "parish_code": "string | null"
  }
}
```

Error response

- HTTP 400, 404, 500, or propagated upstream status code
- Body model: GatewayResponse
- result shape: ThirdError

Notes

- If third_id is empty or whitespace, the endpoint returns HTTP 400
- If Postman sends an unresolved variable such as {{third_id}}, the endpoint returns HTTP 400

20. GET /queues/connectors/{connector_name}

Gets queued messages from a Gestiona connector queue.

Before retrieving messages, the gateway verifies that the connector exists and that there is an active subscription for it. If no active subscription exists, the gateway subscribes to the connector queue and then retrieves queued messages.

Route parameters

- connector_name required

Query parameters

- operationId optional

Request body model

- none

Upstream calls

1. GET /connectors

- The gateway checks the upstream content array for an item where code equals connector_name.
- If no connector is found, the endpoint returns HTTP 404.

2. GET /queues/subscriptions

- The gateway checks the upstream content array for an item where name equals connectors#{connector_name}.

3. POST /queues/connectors/{connector_name}/subscription

- Called only when no active subscription is found.
- If the upstream response body contains a description property, that value is used as the gateway error result.message.

4. GET /queues/connectors/{connector_name}

- Retrieves queued connector messages.

Access token behavior

This endpoint always uses the configured Gestiona access token from `Gestiona:AccessToken`. It does not use `X-User-Access-Token` from the request headers.

Success response

- HTTP 200 OK
- Body model: `GatewayResponse`
- result shape: array of `QueueConnectorMessageResult`

Success example

```
{
  "operationId": "op-01",
  "success": true,
  "result": [
    {
      "message_id": "9376d53a-176e-4716-beaa-bd4486561bbd",
      "date_signed": "2026-09-03 10:07:09"
    }
  ]
}
```

Error response

- HTTP 400, 404, 500, or propagated upstream status code
- Body model: `GatewayResponse`
- result shape: `QueueError`

Error example

```
{
  "operationId": "op-01",
  "success": false,
  "result": {
    "code": 412,
    "name": "Precondition Failed",
    "kind": "Upstream",
    "message": "already exists other consumer subscribed to queue"
  }
}
```

Notes

- If `connector_name` is empty or whitespace, the endpoint returns HTTP 400.
- If Postman sends an unresolved variable such as `{{connector_name}}`, the endpoint returns HTTP 400.
- The active subscription name must match `connectors#{connector_name}`.
- Each upstream queue message `payload.target` is returned as `message_id`.
- Each upstream queue message entry value is formatted with `DateTimeHelpers.FormatUnixTimestamp` and returned as `date_signed`.

21. GET /queues/connectors/{connector_name}/{message_id}

Gets a message from a Gestiona connector queue.

Before retrieving the message, the gateway verifies that the connector exists and that there is an active subscription for it. If no active subscription exists, the gateway subscribes to the connector queue and then retrieves the requested message.

Route parameters

- `connector_name` required

- `message_id` required

Query parameters

- `operationId` optional

Request body model

- none

Upstream calls

1. GET `/connectors`
 - The gateway checks the upstream content array for an item where code equals `connector_name`.
 - If no connector is found, the endpoint returns HTTP 404.
2. GET `/queues/subscriptions`
 - The gateway checks the upstream content array for an item where name equals `connectors#{connector_name}`.
3. POST `/queues/connectors/{connector_name}/subscription`
 - Called only when no active subscription is found.
 - If the upstream response body contains a `description` property, that value is used as the gateway error `result.message`.
4. GET `/queues/connectors/{connector_name}/{message_id}`
 - Accept: `application/vnd.gestiona.queues.message`

Access token behavior

This endpoint always uses the configured Gestiona access token from `Gestiona:AccessToken`. It does not use `X-User-Access-Token` from the request headers.

Success response

- HTTP 200 OK
- Body model: `GatewayResponse`
- result shape: `QueueConnectorMessageResult`

Success example

```
{
  "operationId": "op-01",
  "success": true,
  "result": {
    "message_id": "9376d53a-176e-4716-beaa-bd4486561bbd",
    "date_signed": "2026-09-03 10:07:09"
  }
}
```

Error response

- HTTP 400, 404, 500, or propagated upstream status code
- Body model: `GatewayResponse`
- result shape: `QueueError`

Error example

```
{
  "operationId": "op-01",
  "success": false,
  "result": {
    "code": 412,
    "name": "Precondition Failed",
    "kind": "Upstream",
    "message": "already exists other consumer subscribed to queue"
  }
}
```

Notes

- If `connector_name` or `message_id` is empty or whitespace, the endpoint returns HTTP 400.
- If Postman sends an unresolved variable such as `{{connector_name}}` or `{{message_id}}`, the endpoint returns HTTP 400.
- The active subscription name must match `connectors#{connector_name}`.
- The upstream queue message `payload.target` is returned as `message_id`.
- The upstream queue message entry value is formatted with `DateTimeHelpers.FormatUnixTimestamp` and returned as `date_signed`.

22. POST /queues/connectors/{connector_name}/{message_id}

Sends a response for a Gestion connector queue message.

Before sending the response, the gateway verifies that the connector exists and that there is an active subscription for it. If no active subscription exists, the gateway subscribes to the connector queue and then sends the response.

Route parameters

- `connector_name` required
- `message_id` required

Query parameters

- `operationId` optional

Request body model

- `SendQueueConnectorResponseRequest`

Request body example

```
{
  "result_success": "FALSE",
  "message": "Mensaje de respuesta al conector"
}
```

Upstream calls

1. GET /connectors
 - The gateway checks the upstream content array for an item where `code` equals `connector_name`.
 - If no connector is found, the endpoint returns HTTP 404.
2. GET /queues/subscriptions
 - The gateway checks the upstream content array for an item where `name` equals `connectors#{connector_name}`.
3. POST /queues/connectors/{connector_name}/subscription
 - Called only when no active subscription is found.
 - If the upstream response body contains a `description` property, that value is used as the gateway error `result.message`.
4. POST /queues/connectors/{connector_name}/{message_id}
 - Content-Type: `application/vnd.gestiona.connector-response+json`
 - Sends `result_success`.
 - Sends message only when it is present.

Access token behavior

This endpoint always uses the configured Gestion access token from `Gestion:AccessToken`. It does not use `X-User-Access-Token` from the request headers.

Success response

- HTTP 200 OK

- Body model: GatewayResponse
- result shape: QueueConnectorResponseResult

Success example

```
{
  "operationId": "op-01",
  "success": true,
  "result": {
    "connector_name": "sigma-medidata-file-docs",
    "message_id": "9376d53a-176e-4716-beaa-bd4486561bbd"
  }
}
```

Error response

- HTTP 400, 404, 500, or propagated upstream status code
- Body model: GatewayResponse
- result shape: QueueError

Error example

```
{
  "operationId": "op-01",
  "success": false,
  "result": {
    "code": 412,
    "name": "Precondition Failed",
    "kind": "Upstream",
    "message": "already exists other consumer subscribed to queue"
  }
}
```

Notes

- The gateway accepts Content-Type: application/json and Content-Type: application/vnd.gestiona.connector-response+json for this endpoint.
- If connector_name or message_id is empty or whitespace, the endpoint returns HTTP 400.
- If result_success is empty or whitespace, the endpoint returns HTTP 400.
- If Postman sends an unresolved variable such as {{connector_name}} or {{message_id}}, the endpoint returns HTTP 400.
- The active subscription name must match connectors#{connector_name}.